WEEK 4: GREEDY ALGORITHMS

Advanced Algorithms

ROADMAP

1. Greedy Matching on a Tree
2. The Exchange Argument
3. Activity Selection
4. Weighted Activity Selection
5. Making Change

Guiding slogan for today: **behind every greedy algorithm is a theorem.**

FROM DYNAMIC TO GREEDY

DYNAMIC TO GREEDY

Dynamic programming breaks a problem into (overlapping) subproblems of a similar form to the original problem.

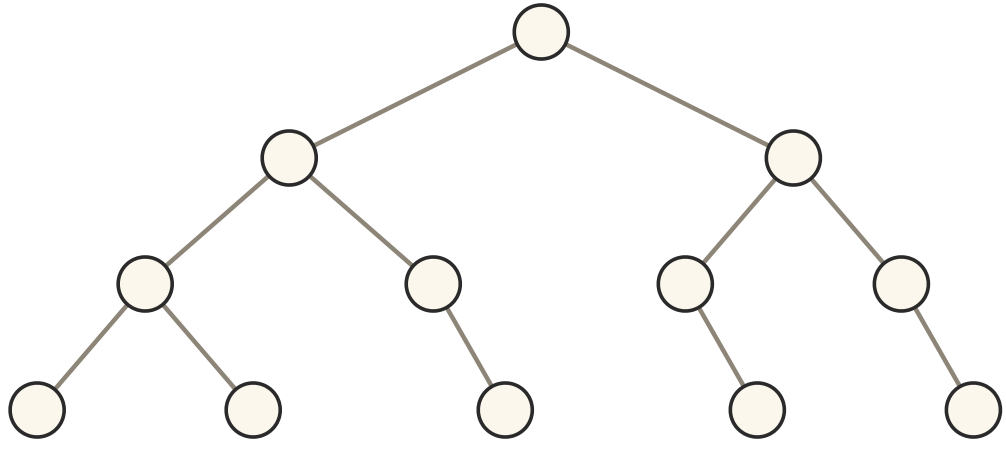
Correctness relies on some kind of **optimal substructure** property.

> **EXAMPLE** A shortest path from A to B also gives shortest paths from A to every intermediate vertex on the path to B.

DYNAMIC CHOICES

In dynamic programming we are faced with several choices and do not know which is best — so we **optimise over all the choices**.

Last week, for maximum matching on a tree, we optimised over taking an edge of the root into the matching or not.



The Week 3 tree: two choices at the root, recurse on both.

GREEDY ALGORITHM

A greedy algorithm is like a dynamic programming algorithm where the choices *collapse*: we **know** the best choice at each step.

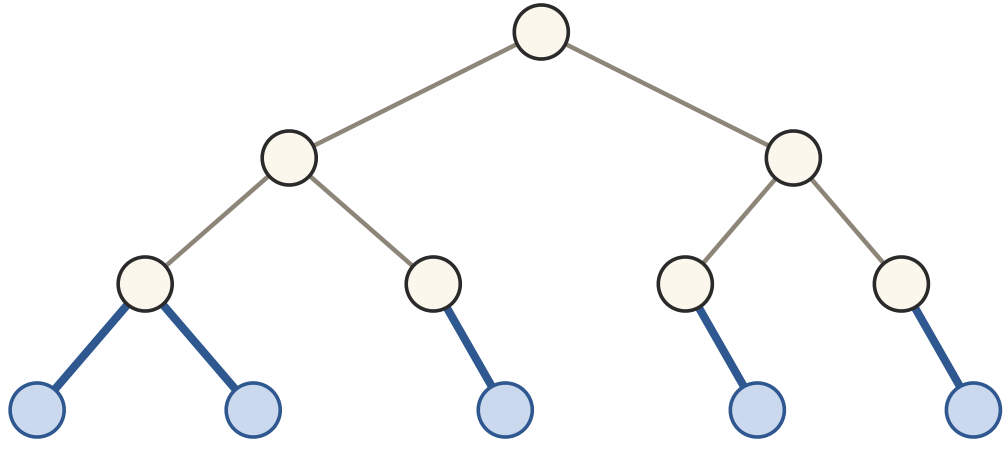
Greedy algorithms can be simpler to code, but harder to come up with.

> **SLOGAN** Behind every greedy algorithm is a theorem — a proof that the greedy choice never hurts.

GREEDY MATCHING ON A TREE

LEAF EDGES

DEFINITION An edge is a **leaf edge** if at least one endpoint has no other edges.



Leaf edges highlighted in blue. Every tree with an edge has a leaf edge.

THE GREEDY ALGORITHM

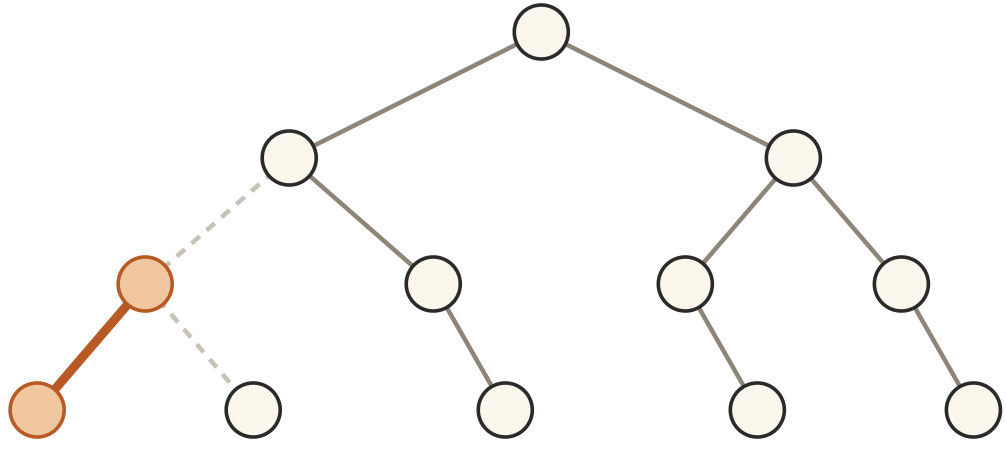
No case analysis, no table — just grab leaf edges:

```
GreedyTreeMatching(T):
    M ← ∅
    while edges remain:
        add a leaf edge to the matching M
        remove all edges incident to the endpoints of the leaf edge
    return M
```

A leaf edge conflicts with other edges at only *one* of its endpoints — intuitively adding it has the least impact on other edges.

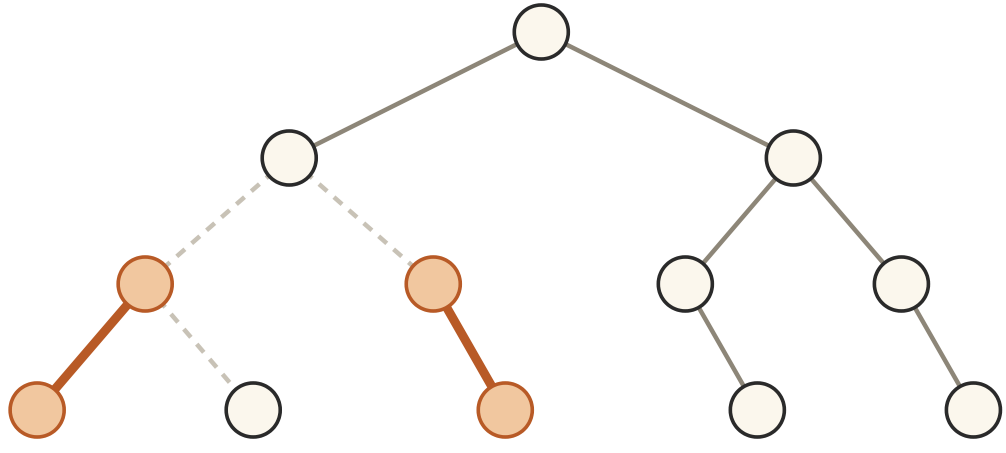
Let's run it.

GREEDY RUN: STEP 1



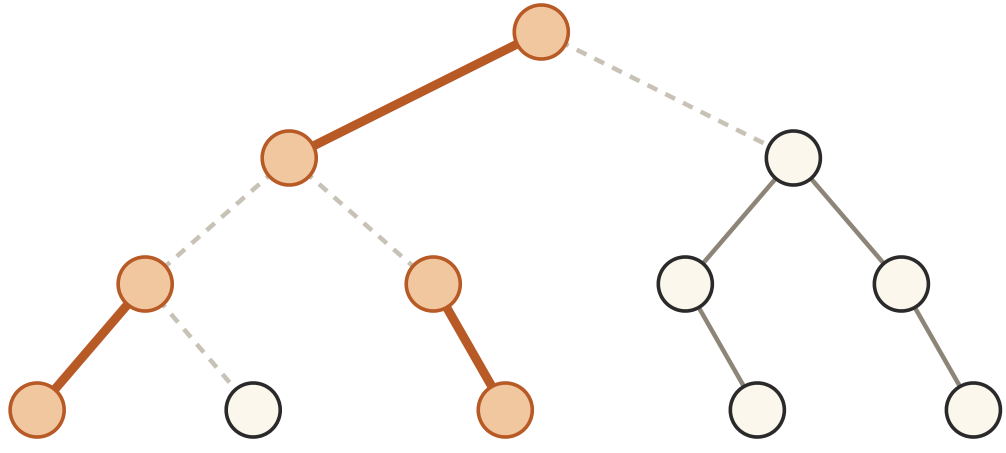
A leaf edge (orange) joins the matching; the edges touching its endpoints are removed (dashed).

GREEDY RUN: STEP 2



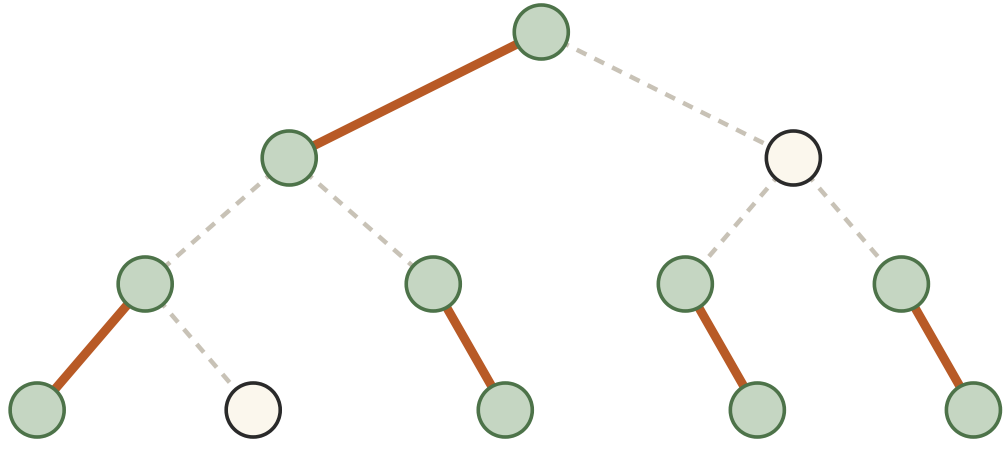
Another leaf edge joins; its parent edge is removed.

GREEDY RUN: STEP 3



Leaf edges are not only at the bottom: after the removals the root–left edge became a leaf edge, and greedy takes it.

GREEDY RUN: DONE



No edges remain.

We obtain a matching of size 5 , which is optimal — and we never branched. But *why* is grabbing leaf edges always safe?

THE EXCHANGE ARGUMENT

EXCHANGE ARGUMENT

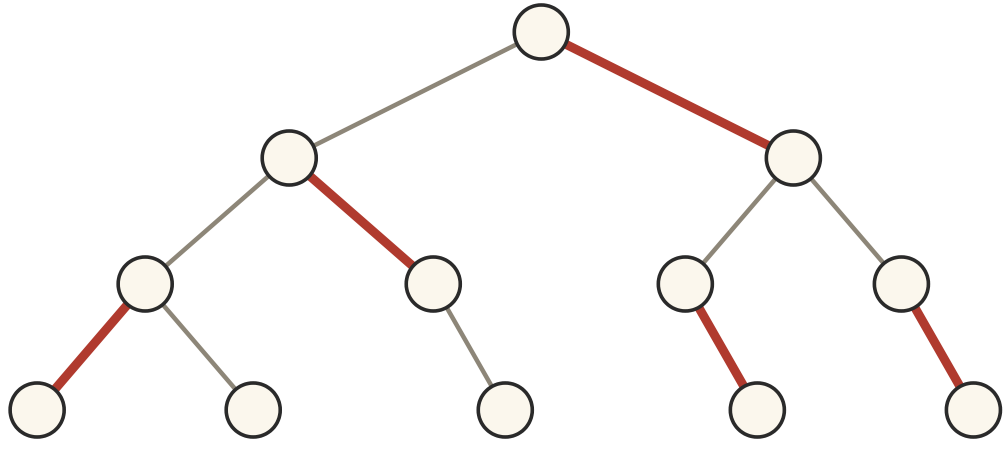
How can we argue that this algorithm finds an optimal solution?

A typical way to argue that a greedy algorithm is correct is an **exchange argument**:

- Start with an *arbitrary* optimal solution.
- Argue that we can, bit by bit, modify it into a solution obtainable by the greedy algorithm, *without making the value worse*.

Then the greedy solution is as good as an optimal one — so it is optimal.

A MATCHING NOT FROM GREEDY



An optimal matching M (red) that was *not* obtained by the greedy algorithm: it uses two non-leaf edges.

Spot the first location where M differs from a greedy run.

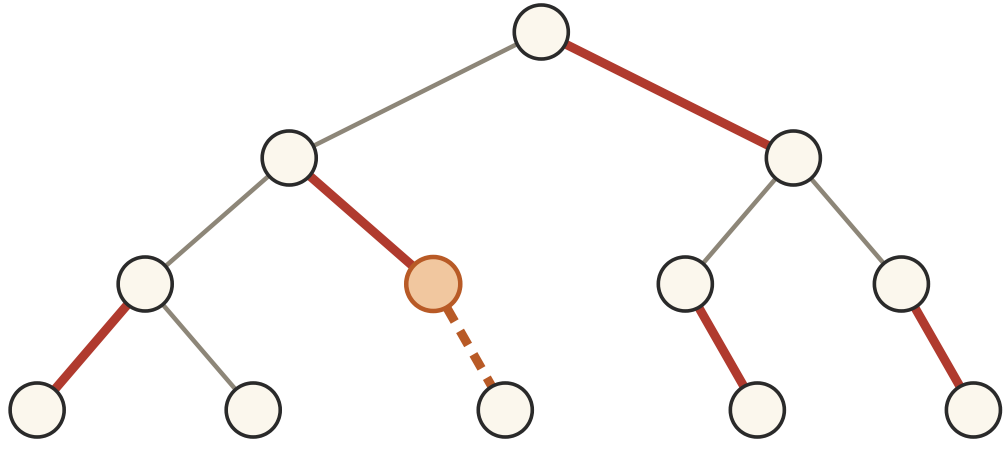
EXCHANGE: CASE 1

> **CASE 1** Suppose some leaf edge could be *added* to M and still leave a matching.

Then we would obtain a bigger matching — contradicting that M is optimal.

So no leaf edge can simply be added: every leaf edge either is in M, or conflicts with an edge of M.

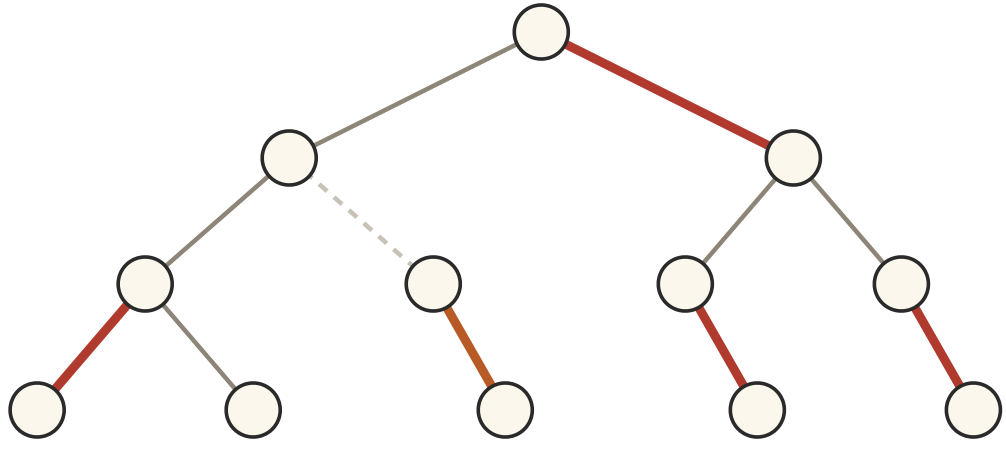
EXCHANGE: CASE 2



Adding the dashed leaf edge creates *one* vertex (orange) with two incident edges — the leaf endpoint had no other edges.

Otherwise: take a leaf edge missing from M whose non-leaf endpoint is matched. Adding it violates the matching at exactly one vertex.

EXCHANGE: THE SWAP



Delete the other edge at that vertex (now dashed): a matching of the *same size* that uses one more leaf edge.

So M is still optimal — and now it agrees with greedy on this leaf edge.

EXCHANGE: RECURSE

The leaf edge is in M. Delete its endpoints and every edge they touch — the algorithm's own deletion step.

What remains of M is a maximum matching of the smaller forest: a bigger one there, plus this leaf edge, would beat M.

Repeat. Each round fixes one more greedy choice and never loses an edge, so M becomes a full run of the greedy algorithm.

BEHIND GREEDY, A THEOREM

> **Theorem** The greedy leaf-edge algorithm outputs a maximum matching on every tree.

Proof shape: *any* optimal solution can be exchanged, one edge at a time, into a run of the greedy algorithm with no loss. Hence $|M_{\text{greedy}}| = |M_{\text{opt}}|$.

Compare with Week 3: the DP also solved this, in $O(n)$ — but it had to consider both choices at every vertex. The theorem is what lets greedy skip the branching.

THE EXCHANGE PROPERTY

ABSTRACT IDEA A greedy choice is **safe** if any optimal solution can be *exchanged*, one swap at a time, into one containing that choice — without making the value worse.

- The argument never exhibits the optimal solution; it shows greedy can *simulate* one, swap by swap.

ACTIVITY SELECTION

ACTIVITY SELECTION

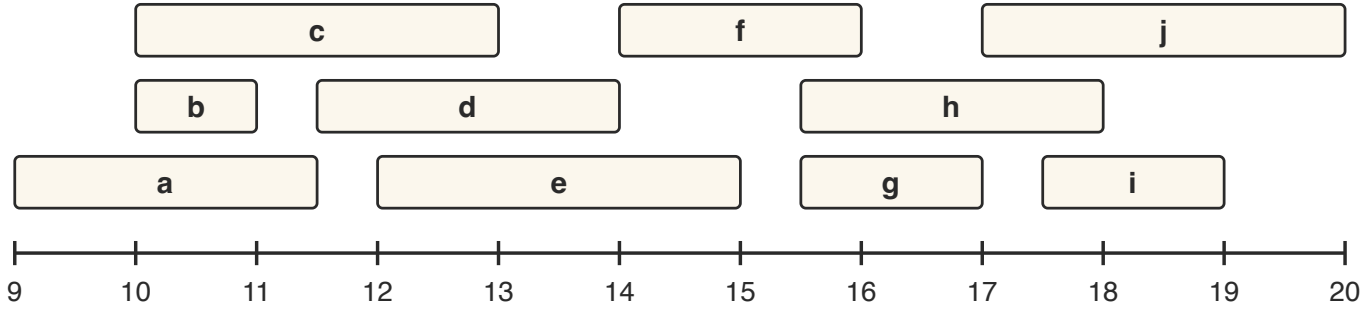
There are a bunch of lectures on Wednesday that we want to attend.

Each lecture has a **start time** and an **end time**.

> **PROBLEM** Choose a schedule where no lectures overlap, attending as **many** lectures as possible.

Also known as *packing intervals*.

WEDNESDAY'S LECTURES



Ten lectures on a time axis; letters name the lectures roughly left to right by start time (a finishes at 11:30, b at 11:00, …, j at 20:00).

How many can we attend?

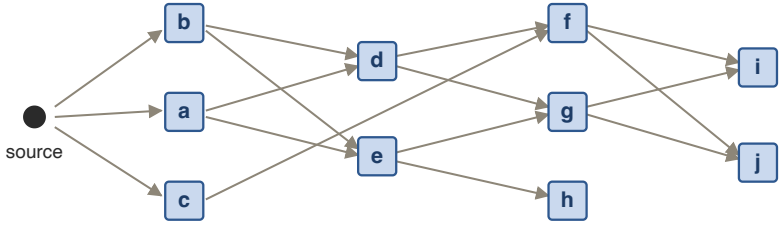
REDUCTION REFLEX: A DAG

We want to *maximize* something — the Week 1 reflex: can we view this as a **longest path in a DAG** problem?

What shall we take as the vertices?

Take one vertex per lecture. What should the edges be?

LONGEST PATH IN A DAG



(not all edges drawn)

EDGES Put an edge (u, v) if $u.\text{finish} \leq v.\text{start}$.

With this definition, the vertices on any path are pairwise **compatible** lectures.

LONGEST PATH IN A DAG

- Add a **source** node with an outgoing edge to every other vertex.
- The longest path from the source is then the maximum number of compatible lectures.

> **CONSEQUENCE** Week 1's longest-path DP already solves activity selection in $O(n^2)$ time (the graph can have $\Theta(n^2)$ edges).

Good — but the graph has special structure. Can we do better?

WHICH VERTEX FIRST?

Which vertex should we visit first from the source?

Intuition: the lecture that **finishes first** leaves the most room for everything else.

Let's turn the intuition into a theorem, in two steps.

FIRST STEP

KEY FACT By the definition of edges, if $u.\,\mathrm{finish} \leq v.\,\mathrm{finish}$ then every out-neighbour of v is also an out-neighbour of u.

Proof: an out-neighbour w of v has $v.\,\mathrm{finish} \leq w.\,\mathrm{start}$, so $u.\,\mathrm{finish} \leq v.\,\mathrm{finish} \leq w.\,\mathrm{start}$.

The earliest finisher *dominates*: whatever any other first lecture can be followed by, it can be followed by too.

FIRST STEP: THE EXCHANGE

Say a longest path goes $\text{source} \to v_1 \to v_2 \to \cdots \to v_k$.

Let u be the lecture that finishes first. Since $u.\,\text{finish} \leq v_1.\,\text{finish}$, the pair (u, v_2) must also be an edge.

So $\text{source} \to u \to v_2 \to \cdots \to v_k$ is also a longest path.

CONCLUSION WLOG the longest path goes to u first: taking the earliest finisher is *safe*. This is exactly an exchange argument.

GREEDY CHOICE PROPERTY

In contrast to dynamic programming, we can **identify** the best choice to make.

We do not have to consider multiple options — the branching in the DP collapses to a single move.

OPTIMAL SUBSTRUCTURE, AGAIN

$\text{source} \to u \to v_2 \to \cdots \to v_k$ is a longest path, so
$u \to v_2 \to \cdots \to v_k$ is a longest path starting from u.

Second step. Consider the lecture x compatible with u that finishes first.
Any out-neighbour of v_2 is also an out-neighbour of x, so
$\text{source} \to u \to x \to v_3 \to \cdots \to v_k$ is also a longest path.

We can apply the exact same argument again!

> **INDUCTION** At every step, choosing the lecture compatible with previous choices that has the **earliest finishing time** extends to a longest path. By induction, greedy builds a longest path — an optimal schedule.

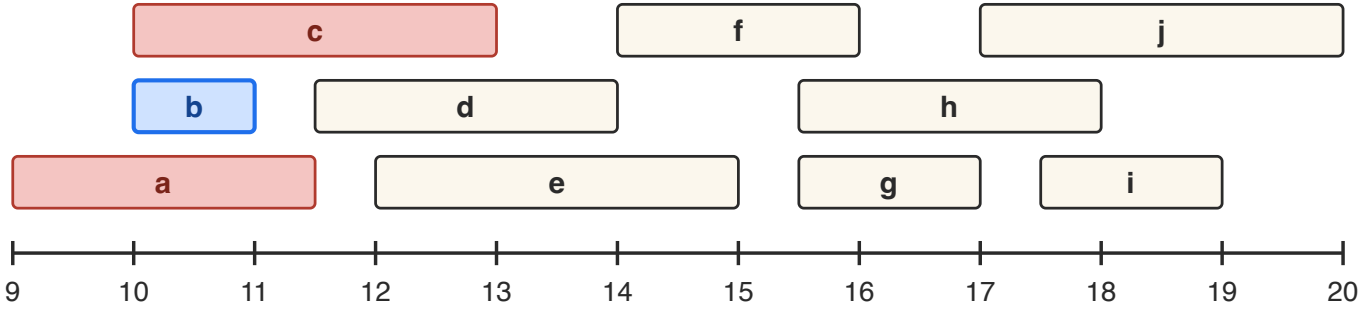
THE GREEDY ALGORITHM

```
ActivitySelection(A):
    sort A by finishing time                    # a₁.finish ≤ a₂.finish ≤ ...
    S ← ∅ ;  last ← −∞
    for each lecture a in sorted order:
        if a.start ≥ last:                      # compatible with all chosen
            S ← S ∪ {a} ;  last ← a.finish
    return S
```

Note we never build the DAG at all — the reduction was scaffolding for the *proof*, not the code.

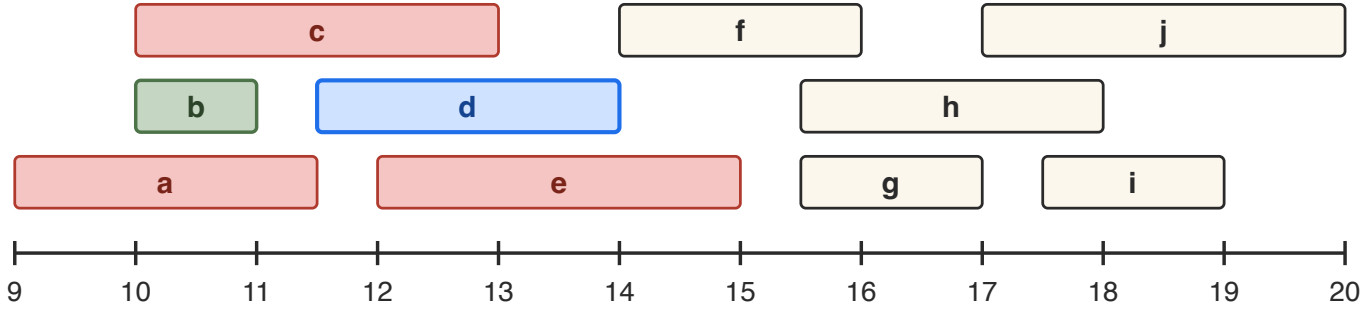
Let's run it on Wednesday's lectures.

GREEDY RUN: PICK b



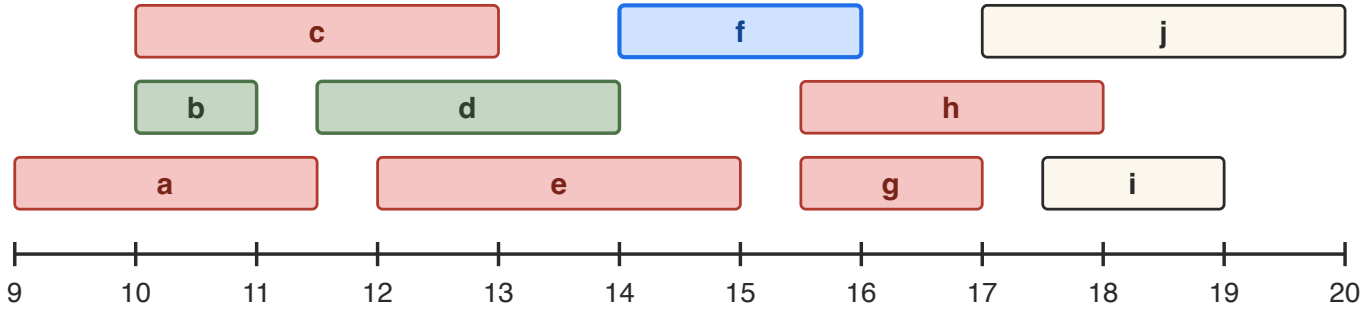
b has the earliest finish (11:00): take it (blue). Lectures a and c overlap it and are ruled out (red).

GREEDY RUN: PICK d



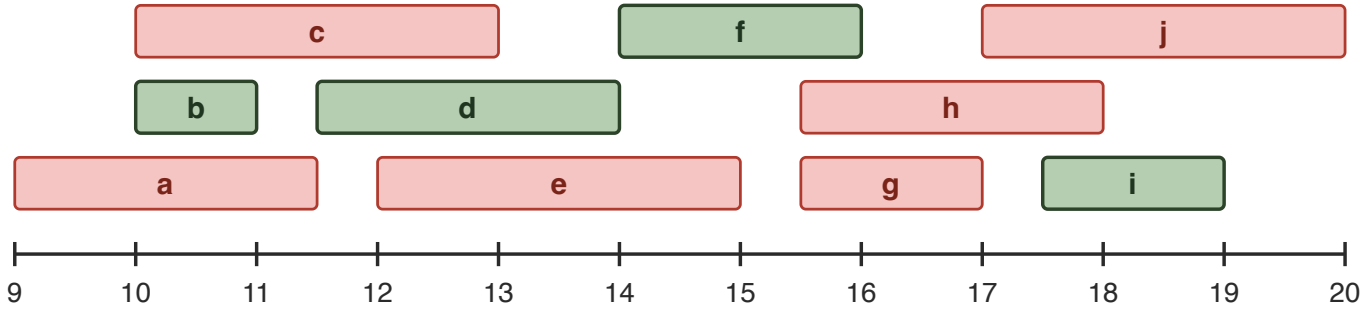
Among lectures starting after 11:00, d finishes first (14:00): take it. e overlaps d and is ruled out.

GREEDY RUN: PICK f



f starts at 14:00 and finishes first among the compatible lectures: take it. g and h start before 16:00 and are ruled out.

GREEDY RUN: PICK i — DONE



Finally i (finishes 19:00) beats j (finishes 20:00). Schedule $\{b, d, f, i\}$ (green): **four lectures**, the maximum.

RUNNING TIME

What is the running time of the greedy algorithm for activity selection?

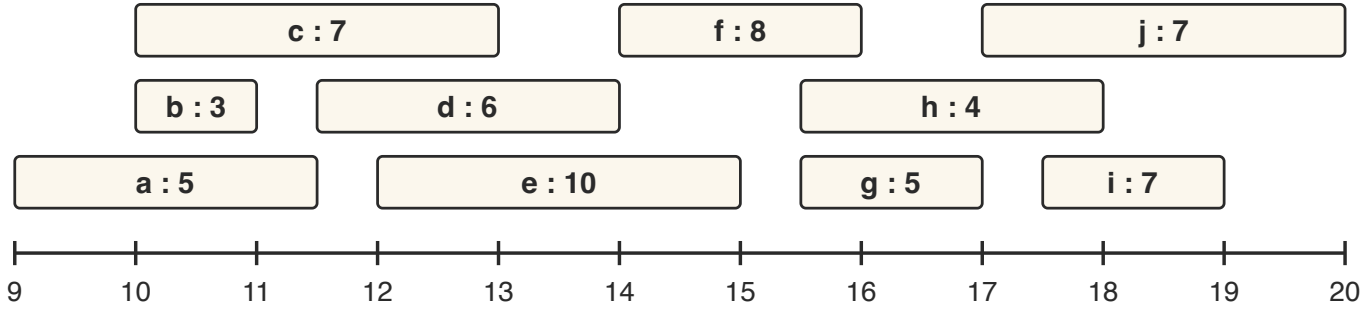
- Sort by finishing time: $O(n \log n)$.
- One pass, comparing each start time against the last chosen finish time: $O(n)$.

TOTAL $O(n \log n)$ — down from the $O(n^2)$ of the longest-path DP. The theorem bought us both simplicity and speed.

WEIGHTED ACTIVITY SELECTION

WEIGHTED ACTIVITY SELECTION

Each lecture now has a **positive enjoyment score**. Choose a schedule with no overlaps that **maximizes total enjoyment**.



Same lectures, now with enjoyment scores.

THE GREEDY CHOICE FAILS

It is no longer obvious that choosing the lecture with the earliest finishing time is best — and in fact it isn't:

Schedule	Lectures	Enjoyment
Earliest-finish greedy	b, d, f, i	$3 + 6 + 8 + 7 = 24$
Optimal	a, e, g, i	$5 + 10 + 5 + 7 = \mathbf{27}$

The exchange breaks: swapping a (score 5) out for the earlier-finishing b (score 3) *does* make the value worse. No theorem, no greedy.

WEIGHTED: BACK TO DP

When the greedy choice collapses, the dynamic choices return. Sort by finishing time. Let $\mathrm{OPT}(i)$ be the best enjoyment of a non-overlapping schedule drawn from lectures $1, \ldots, i$, and let $p(i)$ be the last lecture finishing by the time lecture i starts:

Recurrence

$$\mathrm{OPT}(i) = \max\big(\, \mathrm{OPT}(i-1),\ w_i + \mathrm{OPT}(p(i)) \,\big)$$

— skip lecture i, or take it.

WEIGHTED: RUNNING TIME

Filling the table is $O(n)$ once every $p(i)$ is known. But finding $p(i)$ by scanning back from lecture i for the last finishing time $\leq s_i$ costs $O(i)$ — giving an $O(n^2)$ algorithm.

> **CHALLENGE** Come up with an $O(n \log n)$ time algorithm. (Hint: how fast can you find $p(i)$ in a sorted array?)

MAKING CHANGE

MAKING CHANGE

We are given a set of coin denominations and a target amount.

Problem Given denominations $d_1 < d_2 < \cdots < d_n$ (with $d_1 = 1$) and a target T, find the **minimum number of coins** summing to exactly T. Each denomination may be used as often as we like.

Example: with denominations $1, 5, 25$ and $T = 67$ we can hand over $25 + 25 + 5 + 5 + 5 + 1 + 1$ — seven coins.

What is the dynamic programming algorithm for this?

THE DYNAMIC PROGRAM

The dynamic choice here: *which coin goes into the change first?* We do not know which is best — so we optimise over all of them.

Recurrence Let $C(t)$ be the minimum number of coins summing to t. Then $C(0) = 0$ and

$$C(t) = 1 + \min_{i \,:\, d_i \le t} C(t - d_i).$$

Fill in $C(0), C(1), \ldots, C(T)$ left to right: $O(nT)$ time.

This is the Week 1 reflex again — a *shortest* path in a DAG with vertices $0, \ldots, T$ and an edge $t - d_i \to t$ for every coin d_i.

THE GREEDY ALGORITHM

How does a shopkeeper actually make change?

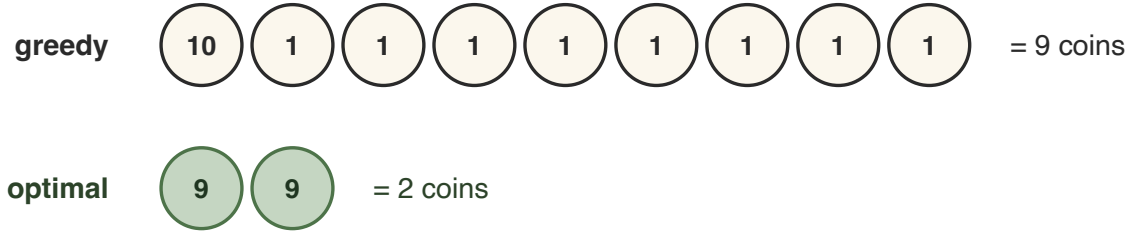
- Start with the largest denomination and take as many of those coins as possible while staying within the target amount.
- Repeat with the remaining amount and the next largest denomination.

On $T = 67$ with denominations $1, 5, 25$: two quarters ($67 \to 17$), three nickels ($17 \to 2$), two pennies — the same seven coins as before, found with no table.

Is this always optimal?

WHEN GREEDY FAILS

This strategy does not always work. Take denominations $1, 9, 10$ and target $T = 18$:



Greedy grabs the 10 and can only finish with eight pennies.

Greedy uses 9 coins where 2 coins suffice — the biggest-looking first step is exactly the wrong move.

WHEN GREEDY WORKS

For other coin systems the greedy strategy *does* always work — for example $\{1, 5, 25\}$. Behind it, as always, a theorem:

> **EXCHANGE** An optimal solution never contains 5 pennies (swap them for a nickel) and never contains 5 nickels (swap them for a quarter).

So in an optimal solution the pennies and nickels together are worth at most $4 \cdot 1 + 4 \cdot 5 = 24$.

WHEN GREEDY WORKS

Pennies and nickels reach at most 24. So for a remaining amount t:

- $t \geq 25$: every optimal solution must use a quarter — greedy's move is forced.
- $5 \leq t \leq 24$: a quarter would overshoot, and at most 4 pennies cannot finish alone — a nickel is forced.
- $t \leq 4$: pennies are all that is left.

Conclusion At every step the greedy choice agrees with *some* optimal solution. By induction, greedy is optimal for $\{1, 5, 25\}$.

WHICH COIN SYSTEMS ARE GREEDY?

There is no simple rule for when the greedy algorithm is optimal for a coin system.

> **THEOREM** Given the n denominations, one can decide in $O(n^3)$ time whether the greedy algorithm is optimal for *every* target amount.

Remarkably, the running time does not depend on the target amounts at all: if any counterexample exists, one exists among just $O(n^2)$ candidate values.

D. Pearson, "A polynomial-time algorithm for the change-making problem", Operations Research Letters 33 (2005).

RECAP

- Greedy = DP where the choices collapse: we *know* the best move.
- Maximum matching on a tree: grab leaf edges, correct by an **exchange argument**.
- Activity selection: earliest finishing time, by exchange plus induction; $O(n \log n)$.

RECAP

- Weighted activity selection: greedy fails, DP gives $O(n^2)$ — $O(n \log n)$ is your challenge.
- Making change: the DP always works; greedy only for special coin systems — and telling which is its own algorithmic problem.
- **Behind every greedy algorithm is a theorem** — certificates of optimality return in Week 10.